CascadeRAG: Abstention-Triggered Escalation that Outperforms Costlier Retrieval-Augmented Generation Architectures at a Fraction of their Cost

Mikołaj Semeniuk[1,*]
Agnieszka Jastrzębska[1]

[1]Faculty of Mathematics and Information Science, Warsaw University of Technology, Koszykowa 75, 00-662 Warsaw, Poland

*Corresponding author: [e-mail address]

Abstract

Retrieval-Augmented Generation (RAG) grounds a language model's answers in an external document collection, and many architectural refinements compete for a place in practical systems. Their published evaluations are rarely comparable, because each uses its own corpus, retriever, generator, and context budget. We present a controlled comparison of thirteen RAG architectures on a single stack (one 21-million-passage corpus, one embedding model, one generator, one prompt) over five question sets and more than 37,000 questions, with paired significance tests under multiplicity control, budget-matched controls, abstention measured separately from error, and retrieval read against the measured ceiling the corpus imposes. No refinement is universally safe: cross-encoder reranking gains +0.051 Exact Match on one question set and loses 0.100 on another. A per-question diagnosis of why retrieval fails explains these reversals and yields a novel design, CascadeRAG: retrieve with a fusion of dense, lexical, and cross-encoder rankings, and escalate to a costlier strategy only when the reader itself declines to answer, a posterior signal that is provably lossless under Exact Match. Across 57 paired comparisons CascadeRAG wins 41, loses 1, and ties 15, at 1.16–1.58 generator calls per question against 2.3–3.0 for iterative and corrective methods, and replicates under a second generator.

Keywords: retrieval-augmented generation; large language models; question answering; information retrieval; evaluation methodology; cascade architectures

1 Introduction

Large language models answer factual questions from knowledge stored in their weights, and that knowledge is incomplete, frozen at training time, and expensive to change. Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) addresses all three problems at once: instead of asking the model to remember, the system searches an external document collection for passages relevant to the question and places them in the model's context, so that the model reads rather than recalls. The technique has become the default way of grounding language models in private, fresh, or specialised document collections, and a large family of architectural refinements has grown around it: lexical–dense hybrid retrieval, cross-encoder reranking, hypothetical document embeddings, corrective retrieval, iterative retrieval interleaved with reasoning, and routers that pick a strategy per question.

The difficulty, for anyone who has to choose among these architectures, is that each refinement is usually evaluated by its authors on their own stack — their own corpus, chunking, retriever, generator, and prompts — and against a small set of baselines. Published numbers are therefore

rarely comparable across papers, and improvements measured on one stack routinely fail to transfer to another. Worse, our measurements show that the refinements are not uniformly safe: the strongest single retrieval improvement we measure, cross-encoder reranking, gains +0.051 Exact Match over the standard baseline on HotpotQA and *loses* −0.100 on NaturalQuestions — a catastrophic regression that a single-benchmark evaluation would never reveal. Practitioners deploying RAG face a menu of architectures whose relative merits depend on the question distribution in ways the literature does not document.

This article makes three contributions.

1. **A controlled, statistically grounded comparison of thirteen RAG architectures.** Every architecture is evaluated on the same 21-million-passage Wikipedia corpus, the same embedding model, the same vector database, the same generator, and the same answer prompt, over five question sets (NaturalQuestions, TriviaQA, HotpotQA, 2WikiMultihopQA, MuSiQue) totalling more than 37,000 questions — so that the only difference between two rows of a results table is the architecture itself. Differences are tested with paired statistics (McNemar's test on Exact Match, Wilcoxon signed-rank on continuous metrics) under a Holm correction, and every retrieval figure is reported against the ceiling the corpus imposes, which we measure. Budget-matched controls separate "a better architecture" from "more text in the context", and abstention is measured separately from error, which reverses one published-style conclusion (Section 7).

2. **A diagnosis of *why* retrieval fails.** Aggregate recall cannot distinguish failure causes that call for entirely different fixes. We classify the baseline's retrieval failures into five causes — wrong slice of the right article, right article ranked too low, article unreachable by the question's own vocabulary, no gold article stating the answer, and article absent from the corpus — and show that the distribution of causes, not any aggregate score, predicts which architectural refinement pays off on which question set.

3. **CascadeRAG, a novel cascade architecture constructed based on lessons learned from the conducted studies.** CascadeRAG combines a fused first stage — dense, lexical, and cross-encoder rankings merged by Reciprocal Rank Fusion, which avoids the reranker's collapse while keeping most of its gain — with *abstention-triggered escalation*: only when the reader itself declines to answer, having seen the retrieved passages, is the question handed to a more expensive strategy. The trigger is a *posterior* signal, unlike routers that predict difficulty from the question alone, and it is *lossless* under Exact Match: across all 887,917 (question, architecture) pairs measured here, no abstention ever scores Exact Match 1, so escalation cannot discard a correct answer. Against every baseline on every question set CascadeRAG wins 41 of 57 paired comparisons, loses 1, and ties 15, at 1.16–1.58 generator calls per question — against 2.3 for iterative retrieval and 3.0 for corrective RAG. The result replicates under a second, unrelated generator.

A fourth, methodological contribution runs through the article: every number reported here — every cell of every table, including those in this sentence — is generated by the benchmark code directly from the per-question output files and imported into the manuscript as a named macro, so no figure can drift from the experiment that produced it. The benchmark, the analysis tools, and the manuscript build are one reproducible pipeline, which we publicly share together with the per-question logs.[1]

The article is organised for two audiences. Sections 2 and 4 are self-contained: they explain how RAG works — embeddings, similarity search, vector databases, and why retrieval is preferred to retraining — and then describe each compared architecture with a uniform template and diagram, so that a reader new to the area can follow the rest. Sections 5–7 present the proposed

[1] `https://github.com/mikolajsemeniuk/ragbench`

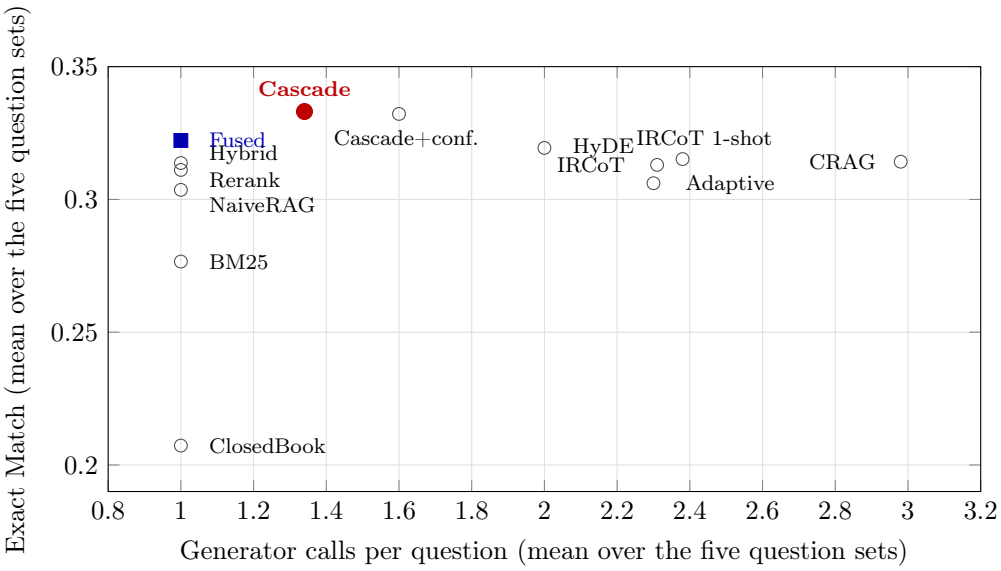


Figure 1: Answer quality against generator cost for every architecture measured in this article, averaged over the five question sets. The proposed cascade (red) reaches the highest mean Exact Match at a fraction of the cost of the iterative and corrective methods; its first stage alone (Fused, blue) already dominates the single-call architectures. Per-set results, in which no single architecture wins everywhere, are analysed in Section 7.

architecture, the experimental protocol, and the results; Section 8 distils practical guidance; Section 9 states limitations.

2 Background: how retrieval-augmented generation works

This section is a self-contained primer. Readers familiar with dense retrieval and RAG can skip to Section 3; every concept introduced here is used later to explain a measured result.

2.1 The knowledge problem, and why not simply retrain the model

A large language model stores what it knows in its weights (Brown et al., 2020; Petroni et al., 2019) — so-called *parametric* knowledge, acquired once, during training. Three consequences follow. First, the knowledge is *frozen*: a model trained in 2024 knows nothing that happened afterwards. Second, it is *incomplete and unreliable*: how much a model retains depends on how often it saw a fact (Kandpal, Deng, Roberts, Wallace, & Raffel, 2023; Roberts, Raffel, & Shazeer, 2020) — popular facts appear thousands of times where rare facts appear once or never (Mallen et al., 2023) — and when asked about the latter it tends to produce fluent, plausible, wrong answers — hallucinations (Ji et al., 2023; Zhang et al., 2023). Third, it is *closed*: an organisation's internal documentation, product catalogue, or case history was never in the training data at all.

The obvious remedy — continue training the model on the missing documents — is rarely practical. Full retraining of even a mid-sized model costs GPU-months; parameter-efficient fine-tuning (Hu et al., 2022) is cheaper but must be repeated every time the documents change, offers no attribution (the model still cannot say *where* an answer came from), and injects new factual knowledge less reliably than simply retrieving it (Ovadia, Brief, Mishaeli, & Elisha, 2023). Retrieval-Augmented Generation (Lewis et al., 2020) sidesteps the problem: the documents stay outside the model, in a searchable index that can be updated by inserting or deleting a record, and at question time the system retrieves the few passages most relevant to the question and prepends them to the model's prompt. The model's job shrinks from *recalling* the fact to *reading*

it. Updating knowledge becomes a database operation rather than a training run; a company can place its private business context into the index without any of it entering the model's weights; and every answer can cite the passages it was read from.

2.2 Embeddings and cosine similarity

The search step must match a question to passages by *meaning*, not just by shared words — "Where did the director of Beat Girl die?" shares almost no vocabulary with the encyclopedia paragraph that answers it. The standard tool is the *text embedding*: a neural network (the *embedding model* or *encoder*, typically a Transformer encoder (Devlin, Chang, Lee, & Toutanova, 2019; Vaswani et al., 2017)) that maps a piece of text to a vector of a few hundred real numbers, trained so that texts with similar meaning land close together in that vector space (Reimers & Gurevych, 2019).

Closeness is measured by *cosine similarity*: the cosine of the angle between two vectors, which is 1 for parallel vectors, 0 for orthogonal ones, and -1 for opposite ones. For vectors $\mathbf{a}$ and $\mathbf{b}$,

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|} = \frac{\sum_i a_i b_i}{\sqrt{\sum_i a_i^2}\sqrt{\sum_i b_i^2}}. \tag{1}$$

A toy example with three dimensions makes the mechanics concrete. Suppose an encoder mapped three texts as follows:

Text	Vector
"Who directed Beat Girl?"	(0.9, 0.4, 0.1)
"Beat Girl is a 1960 film directed by Edmond T. Gréville."	(0.8, 0.5, 0.2)
"The recipe calls for two eggs and a cup of flour."	(0.1, 0.2, 0.9)

The question and the film passage give $\cos = (0.72 + 0.20 + 0.02)/(1.0 \cdot 0.97) \approx 0.97$; the question and the recipe give $\cos \approx 0.35$. The film passage is retrieved. Real encoders behave the same way in 768 dimensions instead of three.

Two properties of embeddings matter for the results in this article. First, an embedding is a *lossy compression*: 768 numbers cannot preserve everything in a hundred-word passage, and what the compression loses first is rare surface forms — exact titles, names, codes. A question that names its target verbatim can therefore still miss it in embedding space, which is precisely the failure that lexical search repairs (Section 4.3). Second, some encoders are trained *asymmetrically*: the query is expected to carry a short instruction prefix (e.g. `query: ...`) that the passage side does not. Ignoring the convention silently degrades retrieval; we verify the convention for our encoder empirically (Section 6).

2.3 Embedding models and cross-encoders

The encoder used for search is a *bi-encoder*: it embeds the question and each passage *separately*, and their relevance is the similarity of the two vectors. This is what makes web-scale search possible — all passage vectors are computed once, offline, and a question at query time is embedded once and compared against millions of stored vectors in milliseconds. The price is that the encoder never sees the question and the passage *together*, so it cannot model their interaction; it matches what a question is *about*.

A *cross-encoder* (Nogueira & Cho, 2019) reads the concatenated (question, passage) pair through one network and outputs a relevance score. Seeing both texts jointly, it can judge whether the passage actually answers the question — what the question is *asking for* — far more precisely, but at the cost of one full network pass per pair, which rules out scanning a corpus. The standard compromise is a two-stage pipeline: a bi-encoder retrieves a shortlist of candidates, and the cross-encoder reorders the shortlist (Section 4.4). The distinction between *about* and *asking for* is not cosmetic; it produces the single largest failure we measure (Section 7.2).

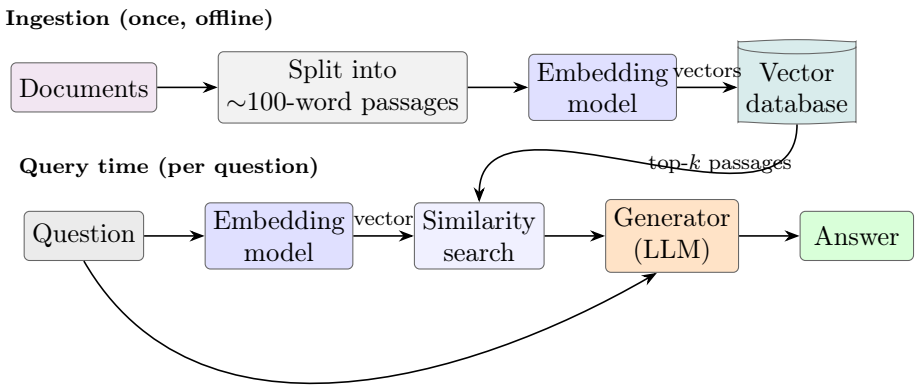


Figure 2: The naive RAG pipeline. Offline, the document collection is split into passages, embedded, and stored in a vector database. At query time the question is embedded with the same model, the k most similar passages are retrieved, and the generator answers from the question plus those passages. Every architecture in this article is a modification of one or more blocks of this pipeline.

2.4 Vector databases

Storing millions of embedding vectors and answering "which stored vectors are most similar to this query vector" is the job of a *vector database*. Comparing the query against every stored vector (exact search) is linear in corpus size; even GPU-accelerated exact search (Johnson, Douze, & Jégou, 2019) is a poor fit for serving interactive queries at the 21-million-passage scale used here. Vector databases therefore build an *approximate nearest neighbour* (ANN) index — most commonly a Hierarchical Navigable Small World graph (HNSW) (Malkov & Yashunin, 2020), a layered proximity graph walked greedily from an entry point towards the query. The walk inspects a small fraction of the corpus and returns the nearest vectors it visited, trading a tunable amount of recall for orders of magnitude in speed; the size of the candidate list the walk keeps (`ef`) controls that trade, and we report and fix it in our setup because a benchmark that leaves it at an unknown default silently measures the index rather than the architecture.

Alongside each vector, the database stores the passage text and its identifier, so a search returns readable passages, ready to be placed in a prompt. The same database can also host a *sparse* (lexical) index, in which a passage is represented by weighted term counts and matching is by exact term overlap under BM25 scoring (Robertson & Zaragoza, 2009) — the modern packaging of classical keyword search, and the complement to dense retrieval used by the hybrid architectures below.

2.5 The naive RAG pipeline

Figure 2 assembles the components into the baseline architecture that every refinement modifies. Offline, the collection is split into passages of roughly one hundred words — small enough that a retrieved passage is mostly about one thing, large enough to carry a fact in context — and each passage is embedded and stored. At query time, the question is embedded, the k most similar passages are retrieved (here $k = 5$), and the generator receives a prompt of the form "Answer the question based on the given documents…" followed by the passages and the question.

This pipeline can fail in one of a few ways: the passage that answers the question is in the corpus but ranked below the top k; it is unreachable because the question's own words are too far from it in embedding space; the right article was found but the wrong hundred-word slice of it was taken; or the answer simply is not in the corpus. Which of these dominates depends

on the question distribution, and measuring that split — rather than guessing it — is what Section 7.3 contributes; the architectures of Section 4 are, in effect, one remedy per failure mode. The generation side has failure modes of its own — readers use passages unevenly by position in the context (Liu et al., 2024), react measurably to irrelevant passages (Cuconasu et al., 2024), and engineering surveys catalogue further failure points across the pipeline (Barnett, Kurniawan, Thudumu, Brannelly, & Abdelrazek, 2024) — but on this benchmark the bulk of the measurable headroom sits in retrieval, which is where the diagnosis looks.

3 Related work

Retrieval-augmented generation. RAG was introduced as a trainable combination of a dense retriever (Karpukhin et al., 2020) and a sequence-to-sequence generator (Lewis et al., 2020), alongside a family of retrieval-augmented language models that condition on retrieved text during pre-training or decoding — REALM (Guu, Lee, Tung, Pasupat, & Chang, 2020), FiD (Izacard & Grave, 2021), RETRO (Borgeaud et al., 2022), kNN-LM (Khandelwal, Levy, Jurafsky, Zettlemoyer, & Lewis, 2020). Contemporary practice replaces the trained combination with a frozen instruction-following LLM reading retrieved passages in its prompt (Ram et al., 2023; Shi et al., 2024), the form studied here. Surveys chart the field's rapid growth and its progression from naive through advanced to modular pipelines (Y. Gao et al., 2023) and, most recently, to agentic systems (Singh, Ehtesham, Kumar, & Khoei, 2025).

Retrieval-side refinements. Classical lexical scoring (Robertson & Zaragoza, 2009) survives inside modern stacks as the complement to dense retrieval, with Reciprocal Rank Fusion (Cormack, Clarke, & Buettcher, 2009) the standard way to merge the two rankings. Cross-encoder reranking (Nogueira & Cho, 2019) re-scores a dense shortlist with a jointly-encoded relevance model. These methods keep one generator call per question and modify only which passages reach it.

Query-side refinements. HyDE (L. Gao, Ma, Lin, & Callan, 2022) searches with a model-drafted hypothetical answer passage instead of the question, bridging the vocabulary gap between questions and encyclopedic text. IRCoT (Trivedi, Balasubramanian, Khot, & Sabharwal, 2023) interleaves one sentence of chain-of-thought reasoning (Wei et al., 2022) with one retrieval per step, so that entities surfaced by reasoning become retrievable queries — the canonical mechanism for multi-hop questions, related in spirit to acting-and-reasoning agents (Yao et al., 2023). Neighbouring iterative schemes decompose the question into explicit sub-questions (Self-Ask (Press et al., 2023)), retrieve whenever the model's next sentence is uncertain (FLARE (Jiang et al., 2023)), or alternate full generation and retrieval rounds (Iter-RetGen (Shao et al., 2023)).

Self-assessing and adaptive systems. Self-RAG (Asai, Wu, Wang, Sil, & Hajishirzi, 2024) trains a model to emit reflection tokens deciding when to retrieve and whether generations are supported. CRAG (Yan, Gu, Zhu, & Ling, 2024) grades retrieved passages with a trained evaluator and triggers corrective web search when the retrieval is judged poor. Adaptive-RAG (Jeong, Baek, Cho, Hwang, & Park, 2024) trains a classifier that routes each question to no-retrieval, single-step, or multi-step pipelines. All three decide how much work a question needs; the cascade proposed here differs in *when* the decision is made — after the reader has seen the evidence (a posterior signal) rather than from the question alone (a prior one) — and in requiring no trained component at all.

Structure-augmented retrieval. GraphRAG (Edge et al., 2024), RAPTOR (Sarthi et al., 2024) and HippoRAG 2 (Gutiérrez, Shu, Qi, Zhou, & Su, 2025) preprocess the corpus into graphs, trees, or memory structures. They occupy a different cost class — their indexing step itself

involves large-scale LLM calls over the corpus — and systematic evaluation finds their advantage concentrated in query-focused summarisation rather than factoid QA (Han et al., 2025); we therefore exclude them from a comparison whose corpus is 21M passages, and note the exclusion as a limitation (see Section 9).

Benchmarks and evaluation. FlashRAG (Jin, Zhu, Yang, Zhang, & Dou, 2024) standardises datasets, corpora, and reimplementations of a dozen RAG methods, and is the source of our corpus and question sets; MultiHop-RAG (Tang & Yang, 2024) benchmarks multi-hop retrieval specifically, BEIR (Thakur, Reimers, Rücklé, Srivastava, & Gurevych, 2021) zero-shot retrieval, and RGB (Chen, Lin, Han, & Sun, 2024) the generation step's robustness to noisy passages; RAGAS (Es, James, Espinosa-Anke, & Schockaert, 2024) offers reference-free evaluation, and best-practice studies tune pipeline components in isolation (X. Wang et al., 2024). Our work differs from toolkit benchmarking in three ways: paired significance testing with multiplicity control instead of point scores; measurement of the corpus-imposed recall ceiling and of abstention as a behaviour distinct from error; and a per-question failure diagnosis that connects each architecture's gain to the failure mode it repairs.

Cascades (the new approach). Deferring expensive computation until a cheap stage proves insufficient has a long history — boosted detector cascades in vision (Viola & Jones, 2001) and cascade ranking in retrieval (L. Wang, Lin, & Metzler, 2011). The architecture proposed here applies the idea at the level of whole RAG strategies, with the reader's own refusal as the deferral signal; to our knowledge the losslessness of that trigger under span-level Exact Match has not previously been stated or exploited.

4 The architectures under study

We compare thirteen architectures: a no-retrieval bottom line, the naive baseline at three context budgets, and nine refinements drawn from the families of Section 3. Figure 3 groups them by the shape of their control flow, following the taxonomy of Jin et al. (2024): *sequential* architectures run the pipeline of Figure 2 once, changing only which passages reach the generator; *branching* architectures add a generation step before retrieval; *conditional* architectures inspect intermediate state and choose among actions; *iterative* architectures loop. Table 1 summarises what each one changes and what it costs. Each architecture below is described with the same template — what it changes, how it works step by step, and what it costs — and each description states the measured failure mode it targets, anticipating the diagnosis of Section 7.3.

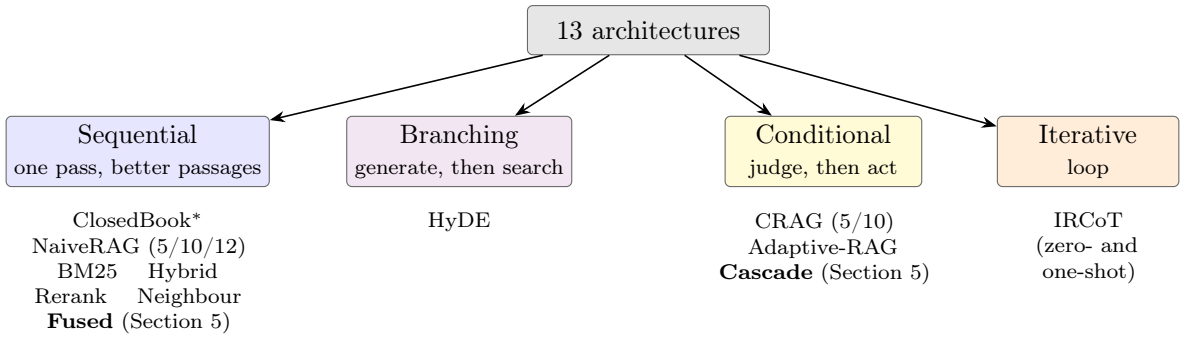


Figure 3: The compared architectures grouped by control flow. *ClosedBook retrieves nothing and serves as a bottom line method for comparisons. The two architectures in bold are introduced in Section 5.

Table 1: Qualitative comparison. "Modifies" names the part of the naive pipeline (Figure 2) that the architecture changes; "Calls" is the number of generator calls per question; "Budget" is the number of passages placed in the generator's context; "Targets" is the retrieval failure cause (Section 7.3) the architecture is meant to repair. The measured per-question costs are in Table 5.

Architecture	Modifies	Calls	Budget	Targets
ClosedBook	removes retrieval	1	0	(bottom line)
NaiveRAG	— (baseline)	1	5	—
NaiveRAG 10/12	context budget	1	10/12	(budget controls)
BM25	retriever (lexical)	1	5	rare surface forms
Hybrid	retriever (dense+lexical)	1	5	rare surface forms
Rerank	ranking (cross-encoder)	1	5	right article ranked low
Neighbour	chunking (expand hits)	1	∼11.5	wrong slice of article
HyDE	query (drafted passage)	2	5	unreachable by query
CRAG	adds retrieval judge	3	5/10	undetected bad retrieval
IRCoT	query (iterative reasoning)	2–6	≤15	multi-hop questions
Adaptive-RAG	routes between strategies	2–7	0–15	heterogeneous difficulty
Cascade (proposed)	retrieval + escalation	∼1.3	≤5	all of the above

4.1 ClosedBook: the bottom line method for comparisons

The generator answers from parametric knowledge alone, with the same answer-only prompt as every other row minus the documents. This is the bottom line each retrieval number must be read against: a 7B model has memorised a large amount of Wikipedia-scale trivia (it answers 0.4241 of TriviaQA without any retrieval), so a high RAG score does not by itself show that retrieval contributed anything. The difference between ClosedBook and NaiveRAG is what retrieval is actually worth on each question set — and on one set that difference will turn out to be an artefact of abstention behaviour rather than knowledge (Section 7.1).

4.2 NaiveRAG: the baseline and its budget controls

The pipeline shown in Figure 2, used without any change: embed the question, retrieve the top five passages, generate. Two further rows run the identical pipeline with ten and twelve passages. These exist because several refinements below place more than five passages in the context, and context size alone is worth measurable Exact Match (naive-10 gains +0.022 over naive-5 on 2WikiMultihopQA); comparing an architecture that reads ten passages against a five-passage baseline would credit the architecture with what is merely a larger budget. Throughout the article, every comparison is made at a matched budget.

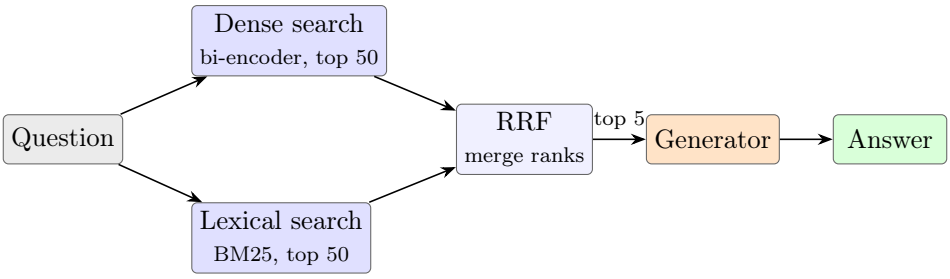


Figure 4: Hybrid retrieval. Dense and lexical rankings over the same corpus are merged by Reciprocal Rank Fusion; the BM25 row is this diagram with the dense branch removed.

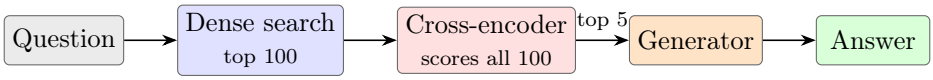


Figure 5: Cross-encoder reranking: a deep bi-encoder shortlist is reordered by a jointly-encoding relevance model; only the reordered top five reach the generator.

4.3 BM25 and hybrid retrieval

The two retrievers fail in different ways. The dense retriever understands paraphrase but blurs rare surface forms (the lossy compression of Section 2.2); the lexical one matches names exactly but is blind to a question that never uses the passage's vocabulary. Hybrid retrieval runs both and merges the rankings with Reciprocal Rank Fusion (Cormack et al., 2009):

$$\mathrm{RRF}(d) \;=\; \sum_{r \,\in\, \text{rankings}} \frac{1}{k + r(d)}, \qquad k = 60, \tag{2}$$

where $r(d)$ is the rank of document d in each list. RRF combines *ranks*, not scores, which is what makes the combination possible at all: a cosine similarity and a BM25 score live on incomparable scales, while their ranks do not. The BM25-only row is the same code with the dense branch disabled, so the difference between the two rows measures exactly "was the dense list fused in" and nothing else. Cost: one extra lexical search on the CPU; the generator is still called once.

4.4 Rerank: cross-encoder reordering

A deep shortlist (100 candidates) is retrieved with the fast bi-encoder and reordered with a cross-encoder (Section 2.3); the top five survivors reach the generator. The target failure is "right article ranked too low": for a third of the diagnosed failures on 2WikiMultihopQA the answer-bearing article is already in the ranking, just below the cut-off, and nothing that reasons over the top five can recover it — the fix has to change which five are chosen. Unlike the iterative methods this adds no generator call; the cost is one cross-encoder pass over the shortlist.

4.5 Neighbour expansion

Wikipedia articles are chunked into approx. 100-word passages with no regard for where a fact ends, so the retriever can land on the paragraph about the right entity while the sentence carrying the answer sits in the next chunk. Neighbour expansion retrieves as NaiveRAG does, then adds, for every hit, the passages adjacent to it in the same article (radius one, capped at fifteen passages, ∼11.5 in practice). It is included in order to *test* the chunking hypothesis, not because our diagnosis supports it: the "wrong slice" bucket is only 5–14% of failures. It is read against the twelve-passage control, because its expanded context would otherwise masquerade as a retrieval gain.

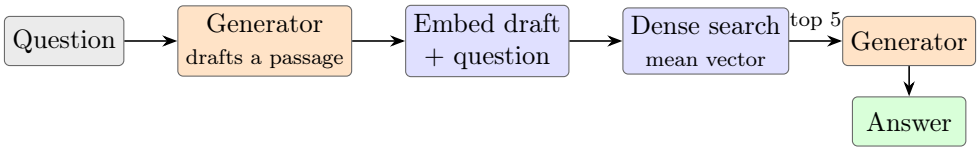


Figure 6: HyDE. The generator first writes the encyclopedia passage it imagines would answer the question; the search runs on the average of the draft's and the question's embeddings. The draft's factual errors are discarded — only its vector is used.

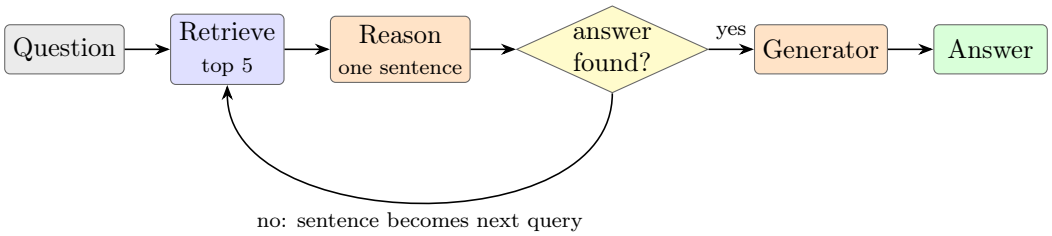


Figure 7: IRCoT. Each reasoning sentence names entities the question never mentioned, and is used verbatim as the next retrieval query; accumulated passages (up to 15) go to the same reader prompt as NaiveRAG.

4.6 HyDE: search with a hypothetical document

A question and a passage that answers it are different kinds of text: "Where was the place of death of the director of film Beat Girl?" shares almost no vocabulary with a paragraph about Edmond T. Gréville. HyDE (L. Gao et al., 2022) removes the mismatch by asking the generator to write the passage it *thinks* would answer the question and searching with that instead. The draft is usually wrong in its details and it does not matter: it only has to look like the right passage so that the encoder places it near the real one. The question's own embedding is averaged in as a safety net, so an off-topic draft degrades towards NaiveRAG instead of chasing a hallucination. This targets the "unreachable by this query" failure bucket — 53% of failures on MuSiQue. Cost: one extra generator call, the cheapest of the query-side methods.

4.7 IRCoT: interleaved retrieval and reasoning

A single query built from the question alone is structurally insufficient for multi-hop questions: "What is the capital of the country where X was born?" never names the country, so no single query can reach the passage that answers it. Measured on the baseline, recall over the context falls from 0.5405 on HotpotQA to 0.3540 on 2WikiMultihopQA to 0.2209 on MuSiQue as the number of required hops grows. IRCoT (Trivedi et al., 2023) alternates: reason one sentence over what has been retrieved, use that sentence as the next query, add what comes back, repeat (up to four rounds or fifteen passages). The final answer is produced by the same reader prompt as NaiveRAG over the accumulated passages, so the only difference between the two systems is the set of passages in context. We run a zero-shot variant and, because the original method prompts the reasoning step with a worked example, a one-shot variant with a single hand-written two-hop demonstration — identical for every question set — as a separate row, so that a under-prompted-baseline objection can be answered with data. Cost: two to six generator calls per question.

4.8 CRAG: corrective retrieval (offline, corpus-only)

NaiveRAG hands the generator whatever came back and hopes it is relevant; the baseline measurements say how often that hope is misplaced (a gold answer appears in the retrieved

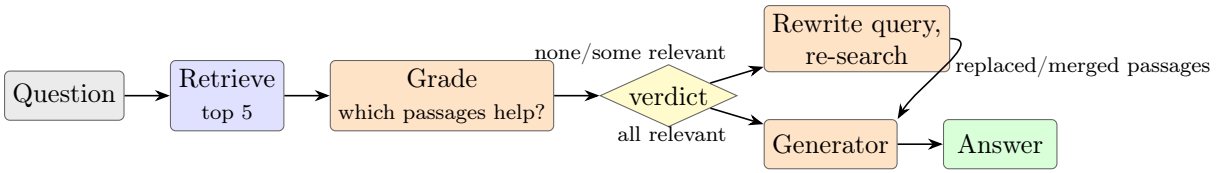


Figure 8: CRAG (offline form). The generator grades its own retrieval; on a poor grade the query is rewritten and the corpus searched again, replacing (no passage relevant) or topping up (some relevant) the context.

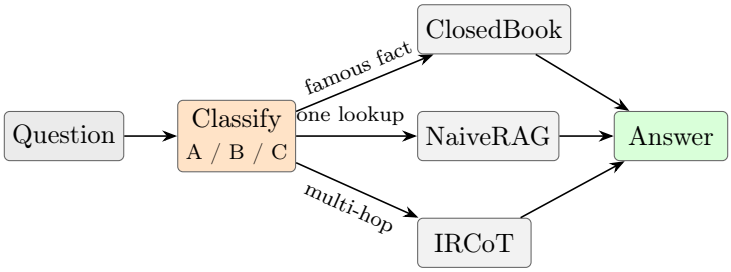


Figure 9: Adaptive-RAG (training-free form). One classification call routes each question to the no-retrieval, single-step, or iterative branch; the branches are the very architectures of the neighbouring rows.

context for only 0.3339 of 2WikiMultihopQA questions). CRAG (Yan et al., 2024) adds the missing step: judge the retrieval before using it, and act when it is bad. Our implementation deviates from the paper in three deliberate, named ways, and is labelled "CRAG (offline, corpus-only)" throughout: the generator itself grades relevance in one call (the paper trains a T5 evaluator; a trained evaluator would confound the controlled comparison with an extra model); the corrective action is a query rewrite over the same corpus (the paper uses web search, which would change the available knowledge, not the architecture); and the paper's knowledge-refinement step is absent. A ten-passage variant pairs with the matched-budget control. Cost: three generator calls per question.

4.9 Adaptive-RAG: routing by predicted difficulty

Every other architecture treats all questions alike, and the measurements say that is wrong in both directions: on TriviaQA the generator already knows 0.4241 of the answers with no retrieval, so running IRCoT over them spends up to six calls to arrive where one would have done, while on MuSiQue a single query reaches the answer-bearing article for barely a fifth of questions. Adaptive-RAG (Jeong et al., 2024) routes each question — here with a single generator classification call rather than the paper's trained classifier, for the same no-extra-models reason as CRAG — to closed-book, single-step, or IRCoT. The router's branches are built from the same components as the architectures they are compared against, so its row is comparable with theirs by construction. Cost: one call plus whatever the chosen branch costs (2.1–3.0 in practice).

5 CascadeRAG: an abstention-triggered cascade

Three measured facts motivate the design. First, *no single architecture wins everywhere*: reranking is the strongest refinement on HotpotQA and the worst failure on NaturalQuestions; HyDE leads on NaturalQuestions and adds little elsewhere; the iterative methods pay two to six calls on every question including the majority that a single call answers just as well. Second, *there is large headroom in choosing per question*: an oracle that picked, per question, the better of closed-book and the naive baseline would gain 0.1087 Exact Match on 2WikiMultihopQA over the better

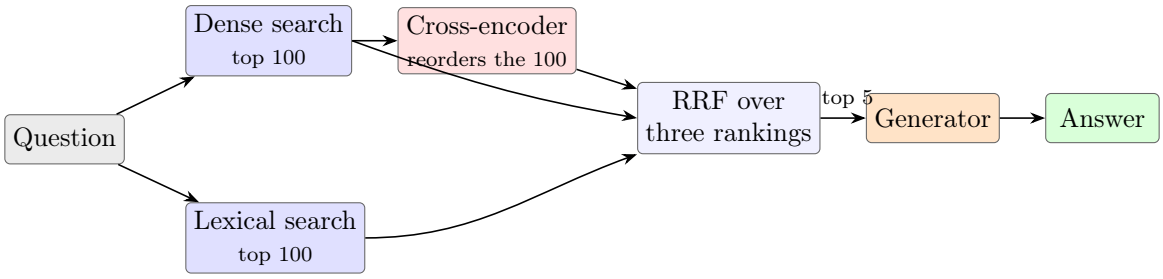


Figure 10: Fused retrieval, the cascade's first stage. The cross-encoder is one voter among three rather than the arbiter: dense, lexical, and reranked orderings of the same candidate pool are merged by RRF (Eq. 2). A passage found by dense retrieval is voted for twice (bi-encoder rank and cross-encoder rank); a passage only the lexical index found is voted for once — the lexical list is there to rescue passages dense retrieval missed, not to outvote it.

single system — an order of magnitude more than any single refinement delivers. Third, *the reader already signals which questions need more work*: a 7B instruct model holding irrelevant passages does not guess; it declines ("the provided documents do not contain…"). On 2WikiMultihopQA the naive baseline declines on 0.1672 of questions, against 0.0223 for the closed-book reader that has nothing to object to. A refusal after reading the evidence is a *posterior* judgement of exactly the quantity a router tries to *predict* from the question alone.

CascadeRAG turns that signal into control flow: answer cheaply, and only when the reader itself says the evidence does not support an answer, try differently. It has two components; in tables and figures the architecture appears as "Cascade".

5.1 Stage one: fused retrieval

The natural first stage would be the strongest single-call architecture, reranking — but its collapse on NaturalQuestions makes it unsafe as a default. The collapse has a mechanism, visible in the per-question dumps. A cross-encoder matches what a question is *asking for* while a bi-encoder matches what it is *asking about*: for "where is the tv show the curse of oak island filmed" the cross-encoder promotes the three shortlist passages that state a filming location — for *Stake Land*, *The Island*, and *Come Outside* — because each answers "where was it filmed" perfectly, and pushes out the passage naming the right show, which says nothing about filming locations. Answer-in-context falls from 0.727 to 0.510 and Exact Match with it (−0.100).

Fused retrieval (Figure 10) therefore demotes the cross-encoder from arbiter to voter: the dense ranking, the cross-encoder's reordering of it, and the lexical ranking are merged with the same RRF as the hybrid baseline. Measured, the fusion is a compromise rather than a free lunch — on NaturalQuestions it scores 0.325 where rerank collapses to 0.2352 (naive: 0.3349); on HotpotQA it scores 0.351 against rerank's 0.3757 — it avoids the catastrophe and gives back part of the gain. Its case is robustness: of the candidate first stages we measure, it has the best mean over the five sets and the smallest worst-case loss (Section 7.6). Cost over plain reranking: one lexical search on the CPU; the generator is still called once.

5.2 Stage two and three: escalation on abstention

Declined questions are handed to HyDE — whose drafted-passage search reaches articles the fused query could not, the dominant remaining failure — and, if HyDE's reader declines too, to closed-book, which at least recovers the guessing behaviour that abstention suppressed. Abstention is detected heuristically: an explicit refusal phrase, or an answer longer than twenty words where the gold answers are short spans; the threshold is a stated hyperparameter.

Two properties distinguish this trigger from prior adaptive systems.

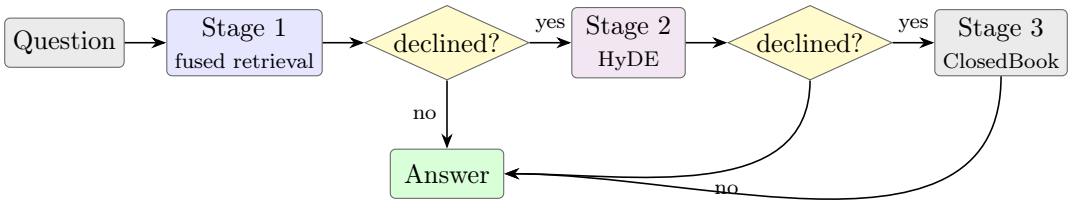


Figure 11: The cascade. A stage's answer is kept unless the reader declined; only declined questions pay for the next stage. The last stage's answer is kept whatever it says — a refusal is still the most honest answer available.

It is posterior. Adaptive-RAG asks the generator to predict, from the question alone, how much work the question will need — and gains +0.0004 Exact Match on 2WikiMultihopQA for it, routing 92% of that multi-hop set down the single-hop branch. The cascade's signal fires *after* the evidence has been seen: it is not a prediction of whether retrieval will fail but a report that it did.

It is lossless under Exact Match. An abstention is a sentence; the gold answers in these datasets are short spans; a sentence cannot normalise to a short span, so an abstention cannot score Exact Match 1 — verified exhaustively over all 887,917 (question, architecture) pairs measured in this study, with zero exceptions. Escalating a declined question therefore *cannot discard a correct answer*: it can only move a question from certainly-wrong to possibly-right. In the vocabulary of selective prediction (Chow, 1970; Geifman & El-Yaniv, 2017), the refusal is a reject option whose selective risk is provably zero under this metric. This is what makes the cascade safe by construction where confidence-threshold deferral is a gamble; it is exact for Exact Match and approximate for token-level F1, where an abstention averages 0.05 and exceeds 0.5 for 0.09% of questions.

The ordering of stages follows their measured cost and coverage: fused answers every question it can at one call; HyDE is the cheapest method that reaches differently (two calls, only for the escalated minority); closed-book costs one call and cannot itself retrieve badly. Only declined questions pay for later stages, which is why the whole cascade averages 1.16–1.58 generator calls per question while containing a two-call and a one-call stage.

5.3 A second, continuous trigger: answer confidence

The abstention trigger is blind to the failure it cannot see: a wrong answer given *without* hesitation. The one free signal about those is the generator's own certainty — a signal language models are reported to carry (Kadavath et al., 2022), though eliciting it reliably is hard (Xiong et al., 2024) — here the mean log-probability of the answer's tokens under greedy decoding, which the serving engine reports at no extra cost. As an extension, the cascade also escalates any answer whose mean token log-probability falls below a threshold τ. Unlike the abstention this is *not* lossless — a correct answer given with low confidence is discarded and only sometimes recovered — so τ is a true hyperparameter, and we select it on 1,000 sampled questions from each *training* split, never on the test sets, by composing independently-run stages question by question offline (exact under greedy decoding, since every stage is deterministic given the question). Whether this second trigger pays at all is an empirical question, answered — negatively — in Section 7.6.

5.4 Reading the cascade's numbers fairly

Three caveats belong in any honest report of this architecture, and we state them before the results. Questions that reach the closed-book stage end with no passages in context, so the cascade's aggregate retrieval metrics are averages over a mixture and are not comparable with a pure retrieval system's; it must be compared on answer quality. Its abstention rate is the *last* stage's, so "Exact Match over questions both systems answered" compares different populations when set against a single-stage system. And the gain, by design, sits in the questions the first stage declined — against reranking, Exact Match restricted to questions both answered is at or below zero on four of five sets. The cascade does not answer better where its first stage already answered; it rescues the questions where the first stage knew it had failed. That is what the design predicts, and the tables of Section 7 should be read with it in mind.

6 Experimental setup

6.1 Unified settings

Every architecture runs against the same stack; the only variable between two rows of any table is the architecture itself.

- **Corpus:** the FlashRAG release of `wiki18_100w` (Jin et al., 2024) — the December 2018 English Wikipedia snapshot chunked into 21,015,324 passages of ∼100 words, the first line of each passage being its article title.
- **Dense index:** `bge-base-en-v1.5` (Xiao, Liu, Zhang, & Muennighoff, 2023) embeddings (768 dimensions, cosine similarity) in Qdrant, HNSW with a fixed search width (`ef` = 256; raised to 1024 where a deeper shortlist is requested, so the graph walk never limits a ranking the architecture needs). For this encoder the documented convention is no instruction prefix on either side; we verified over 500 held-out questions that adding the legacy BGE query instruction changes no retrieval metric beyond its bootstrap interval.
- **Lexical index:** the same corpus and passage ids under BM25 (k_1=1.2, b=0.75) in a separate sparse collection, so the two retrievers can be fused by passage id.
- **Cross-encoder:** `bge-reranker-base` over the dense top-100.
- **Generator:** Qwen2.5-7B-Instruct (Qwen Team, 2024) served by vLLM (Kwon et al., 2023), greedy decoding (temperature 0, so every run is reproducible), 64-token cap on answers, 256 on intermediate generations (a reasoning sentence truncated at the answer's cap would be half a sentence, then used verbatim as a retrieval query). All architectures share one answer prompt ("Answer the question based on the given documents. Only give me the answer…"; Supplementary Material, Section S3), so a score difference measures the passages in context, not answer formatting. The second-reader replication (Section 7.7) swaps in Llama-3.1-8B-Instruct (Grattafiori et al., 2024) and changes nothing else.

6.2 Question sets

Table 2 lists the five sets: two single-hop open-domain sets and three multi-hop sets of increasing compositional depth. Evaluation always runs on the dev or test split; the train splits are used only to tune the cascade's confidence threshold (Section 5.3) and never touch any reported table.

Table 2: Question sets (FlashRAG splits). Recall ceiling is the mean fraction of a question's gold articles present in the corpus at all (Section 6.3); NaturalQuestions and TriviaQA annotate no gold documents.

Question set	Questions	Type	Gold docs	Recall ceiling
NaturalQuestions (Kwiatkowski et al., 2019)	3610	single-hop	no	—
TriviaQA (Joshi, Choi, Weld, & Zettlemoyer, 2017)	11313	single-hop	no	—
HotpotQA (Yang et al., 2018)	7405	2-hop	yes	0.8987
2WikiMultihopQA (Ho, Nguyen, Sugawara, & Aizawa, 2020)	12575	2–4-hop	yes	0.7260
MuSiQue (Trivedi, Balasubramanian, Khot, & Sabharwal, 2022)	2417	2–4-hop	yes	0.8867

6.3 Metrics

Answer quality. *Exact Match* (EM): the normalised answer (lower-cased, punctuation and articles stripped — the standard SQuAD normalisation (Rajpurkar, Zhang, Lopyrev, & Liang, 2016)) equals one of the gold answers; the primary endpoint throughout. *Token-level F1* against the best-matching gold answer. The reader is instructed to output only the answer because the gold answers are short spans; without that instruction an instruct model's correct-but-verbose replies score EM 0 (measured: constraining the output moves EM from 0.02 to 0.40 on a HotpotQA subsample without touching retrieval).

Abstention. EM cannot tell a refusal from a wrong guess, and the two say opposite things about a system. An answer is counted as an abstention on an explicit refusal phrase or when longer than twenty words. We report the abstention rate and EM restricted to answered questions; Section 7.1 shows a ranking that reverses without them.

Retrieval quality. *Recall in context*: the fraction of a question's gold articles among the passages actually placed in the generator's context — deliberately not "Recall@k", because iterative architectures accumulate more than k passages, so the mean context size is reported beside it and the budget-matched controls make comparisons fair. Article titles are compared after Unicode normalisation; the corpus and the question sets disagree on the encoding of accented characters often enough that a byte comparison silently loses 4.5% of 2WikiMultihopQA's gold articles. *MRR*; and *answer-in-context* — whether a gold answer appears as a run of whole tokens in a retrieved passage — for the two sets without gold-document annotation (matching whole tokens, because a substring test scores 0.97 for questions whose gold answer is "no", which occurs inside "north" and "known").

The corpus ceiling. A recall figure is unreadable without the ceiling the corpus imposes: only 0.7260 of 2WikiMultihopQA's gold articles exist in `wiki18_100w` at all, so a perfect retriever would score 0.7260 there, and the baseline's 0.3540 is about half of what is achievable, not a third of it. We measure the ceiling per question set (Table 2) and read every recall against it.

Cost. Generator calls and prompt/completion tokens per question, read from the per-question logs — the only cost measure comparable across hardware. Latency is reported nowhere: at the concurrency the runs used, per-question timing measures the queue, and runs at identical token counts differ in wall-clock by a factor of two depending on what else shared the GPU. The cross-encoder pass and the CPU lexical search are outside the token count and are stated as such where they apply.

6.4 Statistical protocol

Both systems of every comparison answered the same questions, so all tests are paired, which removes question difficulty from the variance — two overlapping confidence intervals can hide a consistent per-question win. Exact Match and the other binary metrics use McNemar's test (McNemar, 1947) on the discordant pairs; continuous metrics use the Wilcoxon signed-rank test (Wilcoxon, 1945); the size of every difference carries a paired bootstrap (Efron, 1979) 95% interval (10,000 resamples, fixed seed). Because a dozen metrics are tested per comparison, p-values are Holm-adjusted (Holm, 1979) within each comparison, with Exact Match the pre-declared primary endpoint. The headline table (Table 4) is stricter still: its claim — "wins W, loses L, ties T of P pairs" — is one family of tests, so Holm runs over every cell of the table at once, and several narrow per-pair wins are ties there.

6.5 Reproducibility

Every run writes one JSON line per question — answer, scores, retrieved passage ids and titles, token counts, and the architecture's internal decisions (which CRAG branch fired, which cascade stage answered, every IRCoT reasoning sentence). All aggregate tables, all significance tests, and all numbers in this manuscript are generated from those logs by the benchmark tools and imported as named macros; no number is typed by hand. One make target per experiment reproduces the corresponding rows end-to-end. The complete pipeline — benchmark code, analysis tools, per-question logs, generated fragments, and the source of this manuscript — is available at `https://github.com/mikolajsemeniuk/ragbench`.

7 Results

We organise the results as six questions. Full per-set tables for every architecture and metric are in Supplementary Material, Section S1; unless stated otherwise, differences quoted below are paired Exact Match differences with Holm-adjusted p-values, and "significant" means $p < 0.05$ after adjustment.

7.1 RQ1: is retrieval worth anything at all?

Against the closed-book bottom line, naive retrieval is worth +0.1950 EM on NaturalQuestions, +0.1458 on TriviaQA, +0.1361 on HotpotQA, and +0.0215 on MuSiQue — large, significant gains everywhere except one set. On 2WikiMultihopQA the naive row appears to *lose* to no retrieval at all (-0.0169 EM), and this apparent inversion is an abstention artefact, not a knowledge difference: holding irrelevant passages, the reader declines on 0.1672 of questions (closed-book: 0.0223), and a decline scores zero where a closed-book *guess* is sometimes right. Restricted to questions both systems answered, the ordering reverses (+0.0113 in naive's favour). Any evaluation that reports EM without abstention would publish the wrong conclusion on this set — which is why every table in this article carries both.

7.2 RQ2: what does each single refinement buy?

Lexical and hybrid retrieval. BM25 alone is well below the dense baseline on the open-domain sets (-0.1100 on NaturalQuestions) but *ahead* of it on 2WikiMultihopQA (+0.0328), whose questions name their target articles verbatim — exactly the rare-surface-form failure of Section 2.2. Fusing the two (Hybrid) is a small but uniformly non-negative improvement (-0.0260 to +0.0410 across sets) at one CPU search of extra cost — the cheapest strictly-safe refinement measured here.

Table 3: Diagnosis of the naive baseline's retrieval failures (300 sampled failures per set; percentages of the diagnosed sample). The last row is the Exact Match an oracle choosing per question between the naive run and closed-book would add over the better single system.

Failure cause	HotpotQA	2Wiki	MuSiQue
Wrong slice of the right article	10.3%	14.0%	7.7%
Right article ranked 11–1000	53.7%	33.7%	41.7%
Unreachable by this query	19.7%	19.3%	35.0%
No gold article states the answer	15.3%	22.3%	15.3%
Gold article not in the corpus	1.0%	10.7%	0.3%
Oracle (retrieve-or-closed-book) gain	0.0516	0.1087	0.0232

Cross-encoder reranking. The strongest single gain and the worst single failure, on different sets: +0.0510 on HotpotQA and +0.0388 on 2WikiMultihopQA, against -0.1001 on NaturalQuestions, where answer-in-context falls from 0.7262 to 0.5097 by the asking-for/asking-about mechanism of Section 5.1. Reranking is a bet on the question distribution, not a safe default.

HyDE. The one refinement that beats the baseline on NaturalQuestions (+0.0127, answer-in-context 0.7262 → 0.7525) and it helps on the multi-hop sets as well (+0.0163 on 2WikiMultihopQA), at the fixed price of doubling the generator calls on every question.

Neighbour expansion. Read against the twelve-passage control, expansion is worth -0.0173 on HotpotQA, -0.0121 on 2WikiMultihopQA and -0.0021 on MuSiQue — a null result, and a predicted one: the diagnosis (RQ3) puts the "wrong slice of the right article" bucket this method repairs at 5–14% of failures. The popular advice to always fetch neighbouring chunks is, on this corpus, worth approximately nothing once context size is controlled.

Iterative and corrective methods. Against the matched ten-passage control, IRCoT gains +0.0109 on HotpotQA, -0.0055 on 2WikiMultihopQA and -0.0033 on MuSiQue at 2.1–2.5 calls per question; the one-shot variant with a worked example changes little (-0.0042 on HotpotQA, +0.0098 on 2WikiMultihopQA), because the zero-shot loop already declares itself able to answer after the first round — the same five passages NaiveRAG reads — on 70–80% of multi-hop questions. CRAG at the same budget gains +0.0149 on MuSiQue, its best set, at three calls per question. Adaptive routing is indistinguishable from its own single-hop branch (+0.0004 vs. naive on 2WikiMultihopQA): predicting difficulty from the question alone sends 92% of a multi-hop set down the single-hop path.

7.3 RQ3: why does retrieval fail?

Table 3 splits the baseline's failures into the five causes of Section 2.5, each demanding a different remedy. Three readings matter. First, the two *unfixable* buckets — answer not stated by any gold article, article absent from the corpus — account for roughly a quarter to a third of failures, which is the concrete form of the recall ceiling. Second, the *ranking* bucket (article present, reachable, ranked 11–1000) is the largest fixable one on HotpotQA and 2WikiMultihopQA — which is why reranking is the strongest single refinement there — while on MuSiQue the *unreachable* bucket dominates at 35.0%, which is why query-side methods are what helps there. Third, the *wrong slice* bucket that neighbour expansion repairs is small everywhere, predicting that method's null result. The diagnosis, made before the cascade was designed, is the design brief the cascade answers: fuse rankings (largest bucket), escalate to a query-side method (second bucket), fall back to no retrieval (the questions where retrieval cannot help but a guess might).

Table 4: The proposed cascade against every baseline on every question set. Each cell is the paired Exact Match difference (cascade minus baseline); **bold** marks a significant cascade win and underline a significant loss under McNemar's test with Holm adjustment over all 57 cells jointly. The cascade wins 41, loses 1, ties 15.

Baseline (calls/q)	NQ	TriviaQA	HotpotQA	2Wiki	MuSiQue
ClosedBook (no retrieval) (1.0)	**+0.202**	**+0.179**	**+0.170**	**+0.033**	**+0.046**
BM25 (1.0)	**+0.117**	**+0.067**	**+0.052**	**+0.017**	**+0.029**
NaiveRAG, 5 passages (1.0)	+0.006	**+0.033**	**+0.033**	**+0.050**	**+0.024**
NaiveRAG, 10 passages (1.0)	–	–	**+0.028**	**+0.026**	+0.011
NaiveRAG, 12 passages (1.0)	–	–	**+0.022**	**+0.027**	+0.013
Hybrid (dense + BM25) (1.0)	**+0.033**	**+0.019**	**+0.021**	**+0.009**	**+0.014**
HyDE (2.0)	-0.006	+0.008	**+0.024**	**+0.033**	+0.009
Rerank (cross-encoder) (1.0)	**+0.107**	+0.007	-0.017	**+0.011**	+0.003
Neighbour expansion (1.0)	–	–	**+0.039**	**+0.039**	+0.015
CRAG (offline), 5 passages (2.9-3.0)	+0.000	**+0.019**	**+0.030**	**+0.045**	+0.001
CRAG (offline), 10 passages (3.0)	–	–	**+0.017**	**+0.025**	-0.004
IRCoT (2.1-2.5)	+0.006	**+0.031**	**+0.017**	**+0.032**	+0.014
Adaptive-RAG (2.1-3.0)	+0.006	**+0.033**	**+0.029**	**+0.049**	**+0.017**
Cascade calls/q	1.44	1.16	1.22	1.32	1.58

7.4 RQ4: the cascade against every baseline

Table 4 is the headline result: 41 significant wins, 1 loss, 15 ties over 57 (baseline, question set) pairs, under the strictest correction. Against the standard baseline the cascade gains +0.0333 on TriviaQA, +0.0334 on HotpotQA, +0.0497 on 2WikiMultihopQA and +0.0244 on MuSiQue; on NaturalQuestions the difference (+0.0064) is not significant — the one set where the baseline was already near its measured ceiling for this reader. The single loss is to reranking on HotpotQA (-0.0174), the set where the cross-encoder's bet pays; against the same reranking on NaturalQuestions the cascade gains +0.1066. No other architecture in the study has a row without a significant loss; several (rerank, HyDE, CRAG-10) beat the cascade on their single best set. The cascade's claim is not "best everywhere" but *never worse than the baseline, never catastrophic, at near-baseline cost* — and Table 4 is that claim in test form.

Where does the gain come from? The stage distribution says: the fused first stage answers 2989 of 3603 NaturalQuestions questions, 10589 of 11294 on TriviaQA, and 1858 of 2416 on MuSiQue — the escalated remainder splits between HyDE (e.g. 272 on MuSiQue) and closed-book (286). The cascade's own abstention rate collapses to 0.0044–0.0335 (baseline: 0.0796–0.2607), because a question must be declined by *three* strategies to end in a refusal. Consistent with Section 5.4, EM restricted to questions both systems answered is at or below zero against reranking on four of five sets: the cascade wins by rescuing declined questions, not by answering differently where its first stage already answered.

7.5 RQ5: what does it cost?

Table 5 prices every row. The cascade averages 1.34 generator calls and 1.15× the baseline's prompt tokens — against 2–3× calls and 2.2–4× tokens for the corrective and iterative methods it outperforms in Table 4. Figure 1 plots both dimensions: the cascade and its own first stage bound the quality–cost frontier at each end, and every architecture that spends more than 1.6 calls per question sits strictly inside it. Escalation prices the expensive strategies by the questions that need them: HyDE costs two calls *per escalated question*, and only 10–30% of questions escalate.

Table 5: Cost per question, averaged over the sets each architecture ran on, from the per-question logs. The last column is prompt tokens relative to the naive baseline. The cross-encoder pass (rerank, fused, cascade) and the CPU lexical search (BM25, hybrid, fused, cascade) are outside the token count.

Architecture	LLM calls	Prompt tokens	Completion tokens	Passages	× NaiveRAG, 5 passages
ClosedBook (no retrieval)	1.00	66	8	0.0	0.08
BM25	1.00	855	14	5.0	0.99
NaiveRAG, 5 passages	1.00	860	13	5.0	1.00
NaiveRAG, 10 passages	1.00	1685	12	10.0	1.96
NaiveRAG, 12 passages	1.00	2009	11	12.0	2.34
Hybrid (dense + BM25)	1.00	857	12	5.0	1.00
HyDE	2.00	965	158	5.0	1.12
Rerank (cross-encoder)	1.00	858	12	5.0	1.00
Neighbour expansion	1.00	1879	13	11.4	2.18
CRAG (offline), 5 passages	2.94-3.00	1878	37	5.4	2.18
CRAG (offline), 10 passages	3.00	3476	39	10.0	4.04
IRCoT	2.09-2.54	2481	110	6.2	2.88
Adaptive-RAG	2.05-2.96	1410	41	5.4	1.64
Fused retrieval (stage 1 alone)	1.00	857	12	5.0	1.00
Cascade	1.16-1.58	992	38	4.6	1.15

7.6 RQ6: which parts of the cascade matter?

What the fusion adds. Stage one alone against plain reranking: +0.0907 on NaturalQuestions and -0.0237 on HotpotQA — the fusion buys back the collapse at the price of part of the gain, and its mean over the five sets (0.322) is the best of any single-call architecture measured. Swapping the first stage confirms the choice: a rerank-first cascade beats the fused-first one on HotpotQA (-0.0242 for fused-first) but loses +0.0466 on NaturalQuestions; a naive-first cascade is close behind everywhere (+0.0240 on 2WikiMultihopQA) and is the cheapest variant; hybrid-first sits between (+0.0147 on NaturalQuestions). No first stage wins everywhere; fused has the best mean and the smallest worst-case loss, which is its case.

What the escalation adds. The cascade against its own first stage run without escalation: +0.0158 on NaturalQuestions, +0.0112 on TriviaQA, +0.0061 on HotpotQA, +0.0153 on 2WikiMultihopQA, +0.0058 on MuSiQue — one to three points on every set, all significant, for 0.16–0.58 extra calls. The escalation is worth roughly this much on top of *any* first stage we tried; which first stage is best depends on the set, but that the escalation helps does not.

The confidence trigger does not pay. The sweep over 4988 held-out train questions puts the best threshold at -0.150 for a gain of +0.0018 EM over the abstention-only cascade ($p = 0.42$) at +0.25 calls per question. At that threshold on the test sets, the trigger fires on 9–15% of questions and the paired difference against the abstention-only cascade is -0.0008 on NaturalQuestions, +0.0021 on TriviaQA, -0.0020 on HotpotQA, -0.0037 on 2WikiMultihopQA (the one significant difference — a loss) and +0.0000 on MuSiQue, at 0.2–0.36 extra calls. The greedy reader is nearly as confident when wrong as when right: its token log-probabilities carry almost no information the abstention did not already carry. We report this negative result because the opposite is widely assumed; the abstention-only cascade remains the proposed configuration. (Full sweep in Supplementary Material, Section S2.)

The one-shot IRCoT objection. Prompted with a worked example as the original method prescribes, IRCoT changes by -0.0042 on HotpotQA and +0.0098 on 2WikiMultihopQA relative to the zero-shot loop, and the cascade's comparison against it (+0.0217 on 2WikiMultihopQA at 2.4 vs. 1.3 calls) is materially the same as against the zero-shot row. The multi-hop baseline was not under-prompted.

7.7 RQ7: does it survive a different reader?

Everything the cascade claims rests on how a reader behaves when the passages are irrelevant — it must decline rather than guess. One model is not evidence that readers in general do that, so the rows the claims stand on (closed-book, naive, rerank, fused, cascade) were re-run with Llama-3.1-8B-Instruct over the same index and questions, changing nothing else. The pattern transfers: the cascade wins 16 of 20 pairs, loses 2, ties 2 (Holm over the table), with the same signature — large gains over reranking on NaturalQuestions (+0.1043), a small loss to it on HotpotQA (-0.0165), significant gains over the naive baseline on every set, and the escalation worth +0.0086 to +0.0220 over stage one alone. The full second-reader table is in Supplementary Material, Section S1.

8 Discussion: lessons for practitioners and evaluators

The benchmark supports several pieces of guidance that are more general than any single row of the tables.

No retrieval refinement is universally safe — except the boring one. Every advanced method measured here loses to the naive baseline on at least one question set, with one exception: hybrid dense-plus-BM25 fusion, which is small, positive everywhere, and nearly free. A practitioner who can afford exactly one change should make that one. The refinements with headline gains (reranking, HyDE, iterative retrieval) are bets on a question distribution and should be preceded by the diagnosis of Section 7.3 on a sample of the actual workload — the failure-cause split, not the aggregate recall, predicts which bet pays.

Compare at matched budget, or measure context size by accident. Ten passages instead of five is worth +0.02 EM on a multi-hop set before any architectural idea enters. Neighbour expansion looks like a retrieval improvement against the five-passage baseline and dissolves against the twelve-passage control. Any comparison whose systems place different amounts of text in the context is confounded until proven otherwise.

Report abstention, or Exact Match will lie. A reader holding irrelevant evidence declines; a reader holding nothing guesses; EM scores the decline zero and the lucky guess one. That artefact alone reverses the closed-book-versus-RAG ordering on 2WikiMultihopQA. Abstention rate and EM-over-answered belong next to EM in every RAG table.

Posterior signals beat prior predictions. Asking a model whether a question *will* need retrieval (routing) bought +0.0004 EM; acting on the reader's report that retrieval *did* fail bought one to three points on every set at a fraction of the cost of always escalating. The reader's refusal is the cheapest reliable difficulty signal in the system, and it is already being generated — most pipelines simply throw it away.

Confidence is not a second signal — at least not for free. The mean token log-probability of a greedily decoded answer, the most commonly proposed deferral signal, carried no information beyond the abstention it accompanies: tuned on held-out data, it bought +0.0018 EM at +0.25 calls and lost significantly on one test set. Calibration should be measured before a confidence threshold is trusted.

Report the ceiling. Only 0.7260 of 2WikiMultihopQA's gold articles exist in the standard corpus; a quarter of "retrieval failures" there are unfixable by any retriever. Recall reported

without the corpus ceiling overstates the room architectures have to differ — and, we suspect, explains part of the literature's replication noise on this set.

Escalation prices expensive methods correctly. The cascade is not a new retrieval method; it is a pricing scheme. HyDE at two calls per question everywhere is a poor trade on TriviaQA where the fused stage answers 10589 of 11294 questions; HyDE at two calls on the 10–30% of questions the reader declined is close to optimal spend. Any organisation running a RAG stack with a refusal-capable reader can adopt the pattern without training anything: keep the cheap answer unless the reader declined, then escalate.

9 Limitations

One corpus, one domain. All measurements use a single English Wikipedia snapshot chunked at ∼100 words. The failure-cause distribution that motivates the cascade — and therefore the choice of its stages — will differ on enterprise corpora, other languages, and other chunkings; the diagnosis method transfers, the numbers do not.

The abstention detector is heuristic. A refusal is recognised by phrase matching plus a length threshold. Both readers we tested abstain in easily recognised English formulas; a reader fine-tuned never to refuse, or prompted in another language, would starve the trigger, and the cascade would degrade to its first stage. The losslessness property is likewise specific to span-style Exact Match: under token-level F1 it is only approximate, and under long-form or generative evaluation it would not hold.

Two implementation deviations are inherited by the comparison. Our CRAG is offline and corpus-only (no web search, no knowledge refinement) and our Adaptive-RAG classifies with the generator rather than a trained classifier. Both deviations keep the comparison controlled — no row gets an extra model or an extra knowledge source — but they also mean our rows bound those methods' training-free, closed-corpus forms, not the published systems.

Structure-augmented methods are absent. GraphRAG-family methods were excluded because their corpus-preprocessing cost is of a different class at 21M passages. On corpora where that preprocessing is affordable, the comparison should be re-run with them as rows.

Model scale. Both readers are 7–8B instruct models. Larger readers abstain differently — typically less — and shift the closed-book bottom line upward; the cascade's escalation margin at larger scales is unmeasured.

10 Conclusion

We presented a controlled comparison of thirteen retrieval-augmented generation architectures — one corpus, one retriever, one generator, paired statistics, budget-matched controls, and a measured corpus ceiling — together with a per-question diagnosis of why retrieval fails, and an architecture assembled from that diagnosis. CascadeRAG retrieves with a fusion of dense, lexical, and cross-encoder rankings, and escalates a question to a more expensive strategy only when the reader itself, having seen the evidence, declines to answer — a posterior, and under Exact Match provably lossless, trigger. Across 57 paired comparisons it wins 41, loses 1, and ties 15, at 1.16–1.58 generator calls per question, and the result replicates under a second reader. Alongside the positive result, the study documents two useful negative ones: neighbour-chunk expansion

adds nothing once context size is controlled, and answer-token confidence adds nothing over the abstention it accompanies.

The broader claim is methodological. RAG architectures are usually compared as point scores on incompatible stacks; held to the standard of paired tests, matched budgets, abstention accounting, and a corpus ceiling, most published-style gains shrink and one reverses. The architecture that survives that standard is not the cleverest one measured but the one that spends its calls only where a free, reliable signal says they are needed. We release the benchmark, the per-question logs, and the manuscript pipeline (`https://github.com/mikolajsemeniuk/ragbench`) so that both the standard and the architecture can be applied to other stacks as they are.

Acknowledgements

[Acknowledgements, if any.]

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflict of interest

The authors declare no conflict of interest.

Data availability statement

The benchmark code, the analysis tools, the per-question logs of every run, the generated result fragments, and the source of this manuscript and of its Supplementary Material are openly available at `https://github.com/mikolajsemeniuk/ragbench`. The corpus and the question sets are the publicly released FlashRAG versions (Jin et al., 2024).

References

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR).*

Barnett, S., Kurniawan, S., Thudumu, S., Brannelly, Z., & Abdelrazek, M. (2024). Seven failure points when engineering a retrieval augmented generation system. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering (CAIN).*

Borgeaud, S., et al. (2022). Improving language models by retrieving from trillions of tokens. In *Proceedings of the 39th International Conference on Machine Learning (ICML).*

Brown, T. B., et al. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, *33*.

Chen, J., Lin, H., Han, X., & Sun, L. (2024). Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 38).

Chow, C. K. (1970). On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, *16*(1).

Cormack, G. V., Clarke, C. L. A., & Buettcher, S. (2009). Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval.*

Cuconasu, F., Trappolini, G., Siciliano, F., Filice, S., Campagnano, C., Maarek, Y., ... Silvestri, F. (2024). The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval.*

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL).*

Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., ... Larson, J. (2024). From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*.

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, *7*(1).

Es, S., James, J., Espinosa-Anke, L., & Schockaert, S. (2024). RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations.*

Gao, L., Ma, X., Lin, J., & Callan, J. (2022). Precise zero-shot dense retrieval without relevance labels. *arXiv preprint arXiv:2212.10496*.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., ... Wang, H. (2023). Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*.

Geifman, Y., & El-Yaniv, R. (2017). Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems* (Vol. 30).

Grattafiori, A., et al. (2024). The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

Gutiérrez, B. J., Shu, Y., Qi, W., Zhou, S., & Su, Y. (2025). From RAG to memory: Non-parametric continual learning for large language models. *arXiv preprint arXiv:2502.14802*.

Guu, K., Lee, K., Tung, Z., Pasupat, P., & Chang, M.-W. (2020). REALM: Retrieval-augmented language model pre-training. In *Proceedings of the 37th International Conference on Machine Learning (ICML).*

Han, H., Shomer, H., Wang, Y., Lei, Y., Guo, K., Hua, Z., ... Tang, J. (2025). RAG vs. GraphRAG: A systematic evaluation and key insights. *arXiv preprint arXiv:2502.11371*.

Ho, X., Nguyen, A.-K. D., Sugawara, S., & Aizawa, A. (2020). Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics (COLING).*

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, *6*(2).

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR).*

Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics (EACL).*

Jeong, S., Baek, J., Cho, S., Hwang, S. J., & Park, J. C. (2024). Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL).*

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, *55*(12).

Jiang, Z., Xu, F. F., Gao, L., Sun, Z., Liu, Q., Dwivedi-Yu, J., ... Neubig, G. (2023). Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP).*

Jin, J., Zhu, Y., Yang, X., Zhang, C., & Dou, Z. (2024). FlashRAG: A modular toolkit for efficient retrieval-augmented generation research. *arXiv preprint arXiv:2405.13576*.

Johnson, J., Douze, M., & Jégou, H. (2019). Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, *7*(3).

Joshi, M., Choi, E., Weld, D. S., & Zettlemoyer, L. (2017). TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Kadavath, S., et al. (2022). Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*.

Kandpal, N., Deng, H., Roberts, A., Wallace, E., & Raffel, C. (2023). Large language models struggle to learn long-tail knowledge. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., ... Yih, W.-t. (2020). Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Khandelwal, U., Levy, O., Jurafsky, D., Zettlemoyer, L., & Lewis, M. (2020). Generalization through memorization: Nearest neighbor language models. In *International Conference on Learning Representations (ICLR)*.

Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., ... Petrov, S. (2019). Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, *7*.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., ... Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, *33*.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, *12*.

Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *42*(4).

Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., & Hajishirzi, H. (2023). When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*.

McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, *12*(2).

Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint arXiv:1901.04085*.

Ovadia, O., Brief, M., Mishaeli, M., & Elisha, O. (2023). Fine-tuning or retrieval? comparing knowledge injection in LLMs. *arXiv preprint arXiv:2312.05934*.

Petroni, F., Rocktäschel, T., Lewis, P., Bakhtin, A., Wu, Y., Miller, A. H., & Riedel, S. (2019). Language models as knowledge bases? In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Press, O., Zhang, M., Min, S., Schmidt, L., Smith, N. A., & Lewis, M. (2023). Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

Qwen Team. (2024). Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Rajpurkar, P., Zhang, J., Lopyrev, K., & Liang, P. (2016). SQuAD: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Ram, O., Levine, Y., Dalmedigos, I., Muhlgay, D., Shashua, A., Leyton-Brown, K., & Shoham, Y.

(2023). In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics*, *11*.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Roberts, A., Raffel, C., & Shazeer, N. (2020). How much knowledge can you pack into the parameters of a language model? In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, *3*(4).

Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*.

Shao, Z., Gong, Y., Shen, Y., Huang, M., Duan, N., & Chen, W. (2023). Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

Shi, W., Min, S., Yasunaga, M., Seo, M., James, R., Lewis, M., ... Yih, W.-t. (2024). REPLUG: Retrieval-augmented black-box language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Singh, A., Ehtesham, A., Kumar, S., & Khoei, T. T. (2025). Agentic retrieval-augmented generation: A survey on agentic RAG. *arXiv preprint arXiv:2501.09136*.

Tang, Y., & Yang, Y. (2024). MultiHop-RAG: Benchmarking retrieval-augmented generation for multi-hop queries. *arXiv preprint arXiv:2401.15391*.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2022). MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, *10*.

Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2023). Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30).

Viola, P., & Jones, M. (2001). Rapid object detection using a boosted cascade of simple features. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.

Wang, L., Lin, J., & Metzler, D. (2011). A cascade ranking model for efficient ranked retrieval. *Proceedings of the 34th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

Wang, X., et al. (2024). Searching for best practices in retrieval-augmented generation. *arXiv preprint arXiv:2407.01219*.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., ... Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, *35*.

Wilcoxon, F. (1945). Individual comparisons by ranking methods. *Biometrics Bulletin*, *1*(6).

Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2023). C-Pack: Packaged resources to advance general Chinese embedding. *arXiv preprint arXiv:2309.07597*.

Xiong, M., Hu, Z., Lu, X., Li, Y., Fu, J., He, J., & Hooi, B. (2024). Can LLMs express their uncertainty? an empirical evaluation of confidence elicitation in LLMs. In *International Conference on Learning Representations (ICLR)*.

Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective retrieval augmented generation. *arXiv preprint arXiv:2401.15884*.

Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., & Manning, C. D. (2018). HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Zhang, Y., et al. (2023). Siren's song in the AI ocean: A survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*.